7. Add Support for Process Syscall Banning

Now we have what we need in order to do some real work.
Your mission is to add a new functionality to the linux kernel called **process syscall bans**.
The modified kernel will have the option to ban processes from using certain system calls – **getpid()**, **pipe()**, **kill()**, and **signal()**.
Each process can have any combination of these bans, and we would be able to set and check them using system calls. When a process is forked, the child process should inherit all the bans the parent had. Init will have no bans at startup.
To keep the modified kernel as lightweight as possible, **you are forbidden from increasing the size of the PCB struct by more than one byte** (enough for one char sized variable).
Before you start writing code, ask yourselves: how should we store each process bans?
To this end, you will implement four new system calls, as described below.
You are required to implement these four system calls in `kernel/hw2.c` and submit this file, along with any other files you changed in the linux kernel.
We remind you that kernel functions that implement system calls should start with `sys_` , as was demonstrated in the implementation of the `hello()` system call from the previous section.

7.1 `long set_ban(int ban_getpid, int ban_pipe, int ban_kill, int ban_sig)`

System call number: 334.

Action: Sets bans for the current process. Requires root privileges.

Arguments: `ban_getpid` – 1 bans the process from using getpid(), 0 lifts the ban
`ban_pipe` – 1 bans the process from using pipe(), 0 lifts the ban
`ban_kill` – 1 bans the process from using kill(), 0 lifts the ban
`ban_sig` – 1 bans the process from using signal(), 0 lifts the ban

Note: arguments greater than 1 are to be considered as 1.

Return value:
- On success, return 0.
- On failure,
  - `-EINVAL` at least one invalid argument was specified (e.g. negative value).
  - `-EPERM` the calling process does not have root privileges.

Note: the priority of errors corresponds to the order in which they are listed. you should return the error with the highest priority when multiple failures occur.
example - if an invalid argument was specified and the calling process doesn't have root privileges, you should return `-EINVAL`.

We remind you that system calls report failures to user space by returning negative values. In such cases, the `libc` wrapper function will store the absolute return value in the `errno` global variable and return `-1` to the user-level code that invoked the system call.

7.2 `long get_ban(char ban)`

System call number: 335.

Action: Returns true if the current process is banned from using the syscall, or false if it isn't.

Arguments: `ban` – the ban to check, as the syscall's first char (e.g. 'p' for "pipe", lowercase).

Return value:

- On success, returns true (1) if the process is banned from using the syscall represented by 'ban', false (0) if it isn't.
- On failure,
  - `-EINVAL` an invalid ban was specified (any char other than 'g', 'p', 'k', 's').

7.3 `long check_ban(pid_t pid, char ban)`

System call number: 336.

Action: Can be used to check if a process has a specific ban. The calling process must not be banned from using the syscall represented by 'ban' in order to perform the check.

Arguments: `pid` – the pid of the process we want to check.
`ban` – the ban to check, as the syscall's first char (e.g. 'k' for "kill", lowercase).

Return value:

- On success, returns true (1) if the process is banned from using the syscall represented by 'ban', false (0) if it isn't.
- On failure,
  - `-EINVAL` an invalid ban was specified (any char other than 'g', 'p', 'k', 's').
  - `-ESRCH` the target process does not exist.
  - `-EPERM` the calling process is banned from using the syscall 'ban'.

7.4 `long flip_ban_branch(int height, char ban)`

System call number: 337.

Action: Inverts a specific ban of the direct parents of the calling process (father, great father, great great father etc). lifts the ban if it exists or imposes it otherwise.

Arguments: `height` – limits the number of direct parents we'll try to update. In other words, the maximum height we will travel up the family tree of the process to update parents. non-positive values of height should be considered invalid.
`ban` – the ban to flip, as the syscall's first char (e.g. 'g' for "getpid", lowercase).

Return value:

- On success, return the number of direct parents that were banned as a result of this syscall.
- On failure,
  - `-EINVAL` at least one invalid argument was specified.
  - `-EPERM` the calling process is banned from using the syscall 'ban'.

8. Ban Processes from Using Syscalls

8.1 Altering Syscall Implementations

Now that we can track which syscalls are banned, we are ready to implement blocking itself. You now need to find the implementation of the relevant syscalls (getpid, pipe, kill, signal) and alter them so they return –EPERM when the process is banned from calling them. –EPERM must be returned BEFORE any work is done (for example, we don't want to return –EPERM after we have already sent a signal with kill(), or allocated a pipe with pipe()).

The syscalls are found in these files:

```
kernel/sys.c    (sys_getpid())
kernel/signal.c (sys_kill() & sys_signal())
fs/pipe.c       (sys_pipe())
```

Note: If you have trouble finding the syscall implementations, try to use the following site:
https://elixir.bootlin.com/linux/v4.15.12
It allows you to easily search symbols like function names in the linux source code.

Note: You may notice the syscalls are defined using a macro – SYSCALL_DEFINEN.
This macro takes the name of the syscall and its parameters and defines a matching sys_ syscall implementation. because of this, sys_kill will appear as SYSCALL_DEFINE2(kill, …).
You should not use this macro to define your syscalls in hw2.c, as this will create problems.

Note: signal() calls a different syscall under the hood, so blocking sys_signal() won't have an effect. you must find the syscall that really gets called when the user calls signal().
Hint: strace

8.2 Blocking Syscalls Effectively (Bonus)

In this assignment you were tasked with blocking only 4 syscalls. What if we wanted to block more, or maybe whole groups of syscalls? The method of finding each syscall implementation and altering it would prove difficult since we need to find and understand each implementation. We might even break some of the more complex syscalls while trying to alter them, leading to unforeseen consequences for user processes.
In this section, your mission is to find a better way of implementing syscall banning, which doesn't involve modifying system call implementations.
Implementing this feature successfully without modifying syscall implementations permits 10% bonus reward.
Since this is a bonus section, you will have to research ways to do this on your own, without support from the course staff.

Hint: To achieve this, you only have to change one file – arch/x86/entry/common.c.

Important Notes and Tips

- It is important to remember that we do not require anything regarding the runtime complexity of your system call implementations, so there is no need to try and optimize your implementation if it runs correctly. Also bear in mind that you do not control the process tree, meaning that other kernel functions may change it without changing the fields you added.
- The default behaviour of system calls such as `fork` and `execv` already works well with the new features you added, so you don't have to change them.
- In the implementation of `check_ban()`, you will have to find the task_struct (PCB) of a specific process, using it's PID. since you have access to the kernel source code, you can read the implementation of sys_kill() (or any other syscall that find PCB by PID) for ways to do this effectively.
- If you chose not to implement the bonus, you should modify `kernel/sys.c, kernel/signal.c and fs/pipe.c`. If you decide to go for the bonus, you should only modify `arch/x86/entry/common.c` instead.
- The grade calculation formula (with bonus): `grade + 0.1*grade`. 70 becomes 77.
- You are not expected to handle kernel synchronization in HW2.
- We provided you with a simple testing framework for this HW assignment, which comprises the following files:
  - `makefile` - searches the current directory for source files that matches `test*.cxx`, compiles them into `test*.exe` files. `make handin` will create a submission zip from your files (experimental).
  - `hw2_test.h , hw2_test.cxx` – user-level wrapper functions for our system calls.
  - `test1.cxx` - a simple test.

Submission Instructions

You should create a folder containing the following files:

a. The kernel source file `kernel/hw2.c`.
   This file should implement the five system calls described in this assignment: `hello()`, `set_ban()`, `get_ban()`, `check_ban()` and `flip_ban_branch()`.
b. Any other kernel files you might have changed, including but not limited to: `kernel/Makefile, arch/x86/entry/syscalls/syscall_64.tbl` and `include/linux/syscalls.h.`
   Please keep the directory structure as it is in your kernel (for example, put `hw2.c` in the `kernel` directory).
c. If you have chosen to implement the bonus, you should submit `arch/x86/entry/common.c`, and must submit `kernel/sys.c, kernel/signal.c and fs/pipe.c` unchanged (any change to these files will nullify the bonus).
d. A file named `submitters.txt` which includes the ID, name and email of the participating students. The following format should be used:

```
Linus Torvalds linus@gmail.com 234567890
Ken Thompson ken@belllabs.com 345678901
```

Copy the makefile we provided with the assignment into the directory and use `make handin`

This should create a zip that should look like this:

```
final.zip -+
           +- kernel  -+
                       +- hw2.c
                       +- ...
           +- include -+
                       +- linux -+
                                 +- syscalls.h
                                 +- ...
           +- ...
           +- submitters.txt
```

Important Note: when you submit, keep your confirmation code and a copy of the file(s), in case of technical failure. Your confirmation code is the only valid proof that you submitted your assignment when you did.

Don’t forget to smile,
Operating Systems staff